

Обоснование выбора сетки

Сетка параметров подбиралась так, чтобы покрыть как типичные, так и граничные режимы работы in-memory файловой системы, но при этом оставить число запусков практически выполнимым. В основной регулярной части используются значения глубины (D), ширины (W) и заполнения (F):

$$D \in \{2, 5, 10\} \quad W \in \{5, 10, 15\} \quad F \in \{0.3, 0.6, 0.95\}$$

Такая сетка даёт три характерных уровня для каждого измерения: малый, средний и большой. Этого достаточно, чтобы увидеть не только локальные колебания, но и общую тенденцию роста стоимости операций при усложнении структуры дерева.

Выбор узлов 2, 5, 10 по глубине и 5, 10, 15 по ширине объясняется тем, что они отражают качественно разные формы дерева. $D = 2$ описывает почти плоскую иерархию, $D = 5$ соответствует умеренно вложенной структуре, а $D = 10$ уже моделирует глубокие каталожные цепочки, где начинают проявляться накладные расходы на проход по пути. Аналогично, $W = 5$ соответствует узкому дереву, $W = 10$ является промежуточным вариантом, а $W = 15$ позволяет увидеть эффект широких директорий, где возрастают затраты на поиск дочернего узла и обход содержимого. Значения $F = 0.3, 0.6$ и 0.95 покрывают разреженный, средний и почти полностью заполненный режимы.

Помимо регулярной части, в коде добавлены corner-case конфигурации: глубокие и узкие деревья (10, 2), (15, 2), (20, 2), (15, 3) и широкие, но неглубокие деревья (2, 30), (2, 50), (3, 30), (3, 50). Это важно, потому что именно на таких формах сильнее всего расходятся свойства реализаций: одни структуры данных лучше ведут себя на длинных путях, другие на широких каталогах. Следовательно, одной только симметричной регулярной сетки было бы недостаточно.

Логарифмическая шкала при визуализации результатов оправдана тем, что стоимость операций и потребление памяти растут неравномерно и в ряде режимов различаются на порядки. Например, переход от умеренной структуры к очень крупной может резко увеличить число узлов и тем самым сделать различия между реализациями плохо различимыми на линейной шкале. Логарифмическая шкала позволяет одновременно видеть и малые, и большие значения, не теряя информации о поведении на “лёгких” конфигурациях и не сжимая их в почти горизонтальную линию.

Отдельно важна оценка времени экспериментов. Для одной пары “файловая система + профиль нагрузки” регулярная сетка уже даёт $3 \cdot 3 \cdot 3 \cdot 3 = 81$ точку, а с добавлением граничных share-профилей число конфигураций становится ещё больше. При параметрах `repeats = 2` и `ops = 2000` один job на практике занимает минуты, а полный набор запусков по нескольким профилям и нескольким реализациям даёт десятки минут wall-clock времени даже при распараллеливании. Поэтому сетка должна быть достаточно плотной для выявления закономерностей, но достаточно компактной, чтобы оставаться воспроизводимой на обычной машине. В этом смысле выбранная схема является компромиссом между полнотой покрытия и стоимостью вычислений.

Комбинаторный взрыв и необходимость аппроксимации

Проблема комбинаторного взрыва возникает из-за того, что итоговое число рассматриваемых конфигураций растёт как произведение числа вариантов по каждому измерению. Даже если оставить только основные параметры, получаем произведение по глубине, ширине, заполнению, профилю распределения путей, профилю операций, типу

файловой системы, числу повторов и числу операций в прогоне. При добавлении новых уровней хотя бы по одному измерению объём пространства состояний растёт мультипликативно, а не линейно.

В данной задаче полный перебор особенно дорог по двум причинам. Во-первых, каждая точка сетки требует не просто арифметического расчёта, а полноценной генерации синтетической файловой системы, подготовки последовательности операций и последующего прогона на конкретной реализации. Во-вторых, если цель состоит не только в сравнении нескольких заранее заданных конфигураций, но и в рекомендации на произвольных пользовательских параметрах, то заранее промерить все возможные комбинации D , W , F , локальности, распределения и вероятностей операций вообще невозможно: пространство становится слишком большим, а многие параметры по сути непрерывные.

Именно поэтому аппроксимация является единственным практичным путём. Вместо полного перебора всех комбинаций строится суррогатная модель, которая по входным параметрам быстро оценивает ожидаемые метрики: среднюю задержку, хвостовую задержку, память и пропускную способность. Такая модель не обязана идеально воспроизводить поведение системы в каждой точке; её задача в другом: достаточно точно сохранять относительный порядок альтернатив и давать ответ за миллисекунды, а не за минуты или часы реального бенчмарка.

Практический смысл аппроксимации состоит в том, что она переводит задачу из режима “дорогое измерение каждой новой точки” в режим “дешёвый прогноз с выборочной верификацией”. Сначала реальные бенчмарки выполняются на репрезентативной сетке и дают опорные данные. Затем на их основе или на основе инженерных эвристик строится приближённая функция, которая оценивает поведение системы между измеренными точками. После этого пользователь может получать рекомендацию для произвольного набора параметров без запуска полного набора экспериментов.

Таким образом, комбинаторный взрыв здесь не является абстрактной теоретической проблемой, а напрямую ограничивает возможность полного перебора в разумное время. Аппроксимация в такой постановке не просто удобное улучшение, а необходимый инструмент, который делает систему рекомендаций и анализ результатов практически применимыми.